

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**NGUYỄN ĐĂNG KHOA - 52400018
VÕ HỒNG PHÚC - 52400029
THEL EAINdra SOE - 52400344**

HỆ THỐNG NHẮN TIN & GỌI ĐIỆN TRỰC TUYẾN

BÁO CÁO CUỐI KỲ PHÁT TRIỂN PHẦN MỀM FULL-STACK

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**NGUYỄN ĐĂNG KHOA - 52400018
VÕ HỒNG PHÚC - 52400029
THEL EAINdra SOE - 52400344**

HỆ THỐNG NHẮN TIN & GỌI ĐIỆN TRỰC TUYẾN

BÁO CÁO CUỐI KỲ PHÁT TRIỂN PHẦN MỀM FULL-STACK

**Người hướng dẫn
ThS. Vũ Đình Hồng**

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn thầy **ThS. Vũ Đình Hồng**. Trong quá trình làm bài tiểu luận cũng như trong quá trình học trên lớp, thầy đã luôn tận tình chia sẻ những kiến thức chuyên môn quý giá, thầy cũng đưa ra những lời khuyên, lời góp ý cũng như giải đáp những thắc mắc trong suốt quá trình em làm bài tiểu luận. Sự hướng dẫn tận tình của thầy đã giúp em rất nhiều trong việc hoàn thành bài tiểu luận này. Một lần nữa, em xin gửi lời cảm ơn sâu sắc đến thầy

TP. Hồ Chí Minh, ngày 19 tháng 04 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của **ThS. Vũ Đình Hồng**. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 19 tháng 04 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI.....	1
1.1 Vấn đề đặt ra.....	1
1.2 Đối tượng người dùng.....	1
1.3 Mục tiêu và phạm vi.....	1
CHƯƠNG 2. YÊU CẦU HỆ THỐNG.....	1
2.1 Yêu cầu chức năng.....	1
2.2 Yêu cầu phi chức năng.....	2
CHƯƠNG 3. KIẾN TRÚC HỆ THỐNG.....	3
3.1 Kiến trúc tổng thể.....	3
3.2 Xử lý đồng bộ Realtime.....	4
3.3 Xử lý tập tin Media.....	4
3.4 Luồng xử lý Message.....	4
3.5 Luồng xử lý WebRTC & Signaling.....	5
CHƯƠNG 4. THIẾT KẾ CƠ SỞ DỮ LIỆU.....	5
4.1 Sơ đồ ERD.....	5
4.2 Nguyên tắc thiết kế và tối ưu dữ liệu.....	6
4.3 Cơ chế caching và tối ưu truy vấn.....	6
CHƯƠNG 5. HIỆU NĂNG VÀ KHẢ NĂNG MỞ RỘNG.....	6
5.1 Phân phối kết nối.....	6
5.1.1 Xử lý HTTP API.....	7
5.1.2 Xử lý WebSocket.....	8
5.1.3 Xử lý Front-end.....	8
5.2 Đồng bộ realtime với Redis Pub/Sub.....	8
5.3 Minh chứng hệ thống scale.....	9
CHƯƠNG 6. BẢO MẬT.....	11
6.1 Xác thực.....	11
6.2 Bảo vệ hệ thống.....	11
6.3 Bảo mật WebRTC.....	11
CHƯƠNG 7. KẾT LUẬN.....	11
7.1 Kết luận.....	11
7.2 Hướng phát triển.....	12
TÀI LIỆU THAM KHẢO.....	12

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1 Vấn đề

Sự gia tăng nhu cầu giao tiếp thời gian thực đặt ra yêu cầu về một hệ thống có độ trễ thấp, khả năng mở rộng cao và tối ưu tài nguyên. Các mô hình truyền thống như HTTP Polling không còn đáp ứng hiệu quả, đòi hỏi một kiến trúc mới phù hợp hơn cho Real-time Communication (RTC).

Hệ thống được phát triển như một nền tảng giao tiếp thời gian thực có thể triển khai độc lập hoặc tích hợp, phục vụ các ứng dụng cần trao đổi dữ liệu tức thời với độ tin cậy cao.

1.2 Đối tượng người dùng

Hệ thống được xây dựng như một nền tảng giao tiếp thời gian thực có thể triển khai độc lập hoặc tích hợp. Đối tượng sử dụng gồm:

- Người dùng cuối với yêu cầu trải nghiệm mượt mà, độ trễ thấp.
- Quản trị hệ thống với nhu cầu vận hành, giám sát và mở rộng linh hoạt.

1.3 Mục tiêu và phạm vi

Hệ thống hướng đến việc đảm bảo độ trễ thấp trong truyền tải dữ liệu, khả năng mở rộng theo chiều ngang và duy trì tính sẵn sàng cao thông qua kiến trúc stateless kết hợp các cơ chế hỗ trợ phân tán. Giải pháp tập trung vào các chức năng cốt lõi gồm: nhắn tin thời gian thực, quản lý trạng thái người dùng, gọi Audio/Video P2P qua WebRTC và truyền tải tệp.

CHƯƠNG 2. YÊU CẦU HỆ THỐNG

2.1 Yêu cầu chức năng

Dựa trên hiện trạng triển khai thực tế, hệ thống đã phát triển và đáp ứng đầy đủ các yêu cầu chức năng cốt lõi phục vụ cho hoạt động vận hành, bao gồm:

- Xác thực và định danh (Authentication): Cung cấp cơ chế đăng ký và đăng nhập an toàn, đảm bảo định danh người dùng trong toàn hệ thống.
- Nhắn tin thời gian thực (Real-time Messaging): Hỗ trợ truyền tải và đồng bộ tin nhắn văn bản theo thời gian thực với độ trễ thấp, áp dụng cho cả mô hình hội thoại cá nhân (Private Chat 1-1) và hội thoại nhóm (Group Chat).
- Lưu trữ và truy xuất lịch sử (Message Persistence & Retrieval): Triển khai cơ chế tải dữ liệu theo phân trang (Pagination/Lazy Loading), cho phép truy xuất lịch sử tin nhắn một cách hiệu quả, tránh việc tải toàn bộ dữ liệu gây ảnh hưởng hiệu năng.
- Trạng thái hoạt động (Online Presence & Activity Indicators): Đồng bộ trạng thái người dùng (Online/Offline) theo thời gian thực trên toàn hệ thống, đồng thời cung cấp các tín hiệu hoạt động như trạng thái đang nhập liệu (Typing Indicators).
- Chia sẻ đa phương tiện (Media Sharing): Hỗ trợ đính kèm, tải lên và phân phối các tập tin đa phương tiện (hình ảnh, dữ liệu) thông qua hệ thống Cloud CDN nhằm tối ưu tốc độ truy cập và băng thông.
- Theo dõi vòng đời tin nhắn (Message Lifecycle & Read Receipts): Quản lý trạng thái tin nhắn theo từng giai đoạn bao gồm: Đã gửi (Sent), Đã nhận (Delivered) và Đã đọc (Seen), đảm bảo tính minh bạch trong quá trình truyền tải.
- Gọi thoại và video (WebRTC P2P Call): Cho phép thiết lập và duy trì kết nối Audio/Video chất lượng cao thông qua cơ chế WebRTC theo mô hình mạng ngang hàng (Peer-to-Peer), tối ưu độ trễ và băng thông truyền tải.

2.2 Yêu cầu phi chức năng

Trong phạm vi triển khai hiện tại, hệ thống đáp ứng các yêu cầu phi chức năng (Non-functional Requirements) quan trọng, phản ánh qua các tiêu chí sau:

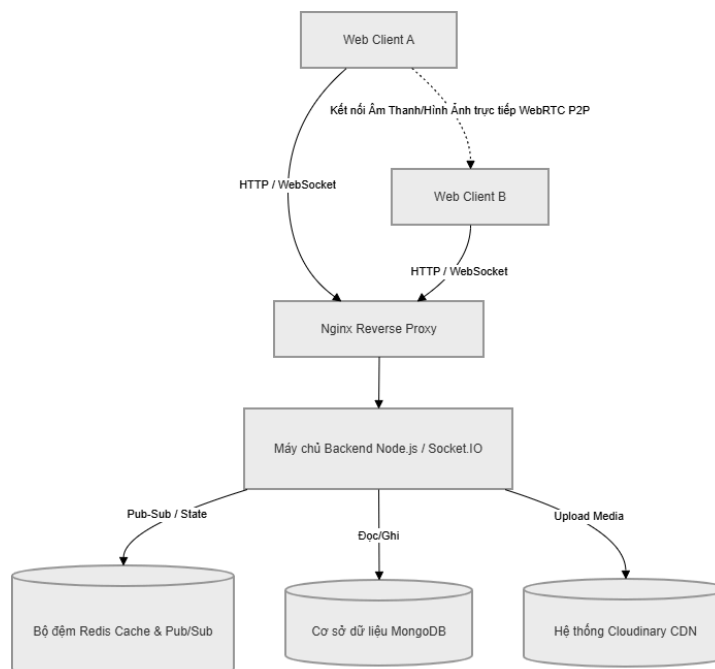
- Hiệu năng và khả năng mở rộng (Performance & Scalability): Cấu trúc Stateless Backend cho phép tăng tải trực tiếp thông qua tính năng scale

backend bằng Docker Compose mà không làm thất thoát tín hiệu (nhờ Redis Pub/Sub).

- Tính khả dụng (Availability): Container quản lý dưới hệ thống Docker sẽ tự động hồi phục qua cấu hình restart: always tích hợp, kết hợp mô hình chia tải cơ bản của Nginx.
- Bảo mật (Security): Kiểm soát lưu lượng qua Rate Limiting (giới hạn Request API để chống Spam), áp dụng băm mật khẩu chuẩn mã hóa Bcrypt và xác thực token bằng JWT.
- Tính bảo trì (Maintainability): Codebase được tổ chức theo hướng Module hóa. Môi trường triển khai được đóng gói bằng Docker, đảm bảo tính nhất quán giữa các môi trường phát triển và vận hành.

CHƯƠNG 3. KIẾN TRÚC HỆ THỐNG

3.1 Kiến trúc tổng thể



Hệ thống được thiết kế theo mô hình scalable architecture:

- Nginx đóng vai trò Load Balancer + Reverse Proxy

- Backend Node.js hoạt động theo kiến trúc Stateless
- MongoDB lưu trữ dữ liệu dài hạn
- Redis xử lý dữ liệu tạm thời và Pub/Sub

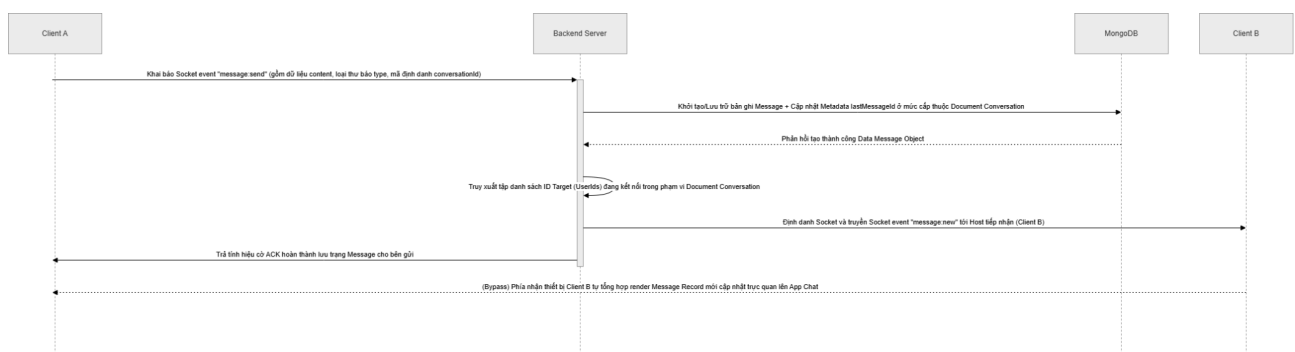
3.2 Xử lý đồng bộ Realtime

Thành phần Realtime được quản lý bằng namespaces và rooms của Socket.IO để tách biệt ngữ cảnh giao tiếp. Hệ thống chia thành các module: chatHandler (xử lý message), presenceHandler (cập nhật heartbeat lên Redis), và callHandler (trung chuyển SDP/ICE cho WebRTC). Mỗi Node.js process sử dụng Redis Adapter để đồng bộ trạng thái và đảm bảo State Isolation trong môi trường phân tán.

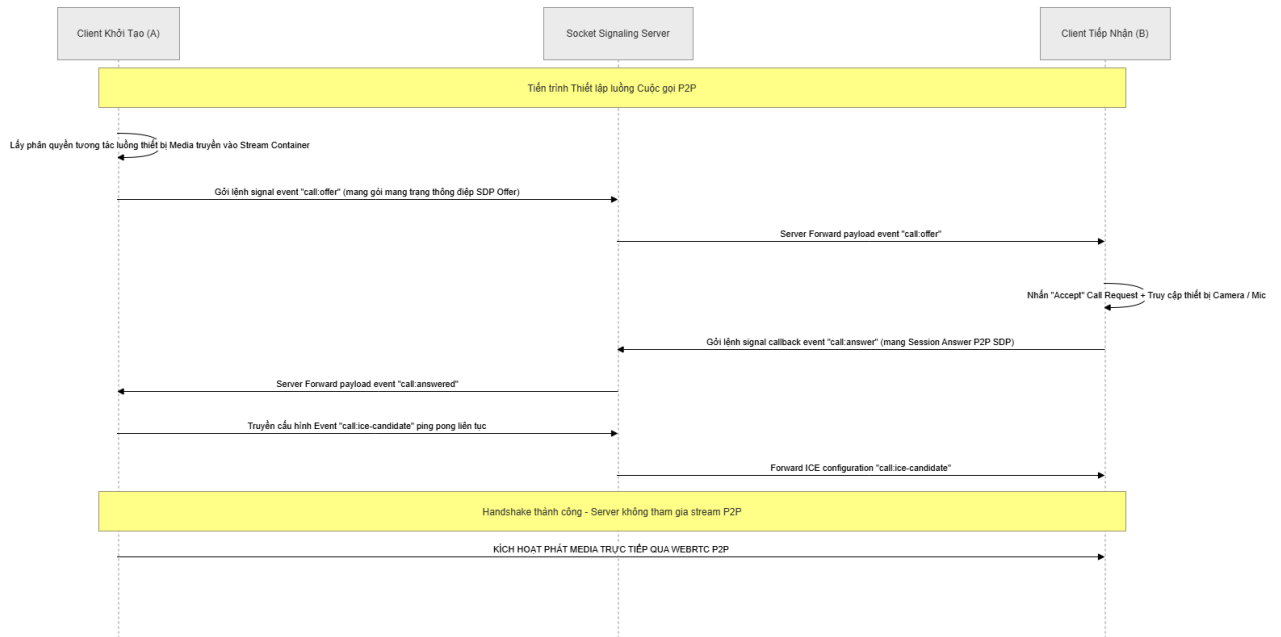
3.3 Xử lý tập tin Media

Tuân thủ nguyên tắc không blocking Event Loop, hệ thống không truyền Binary Data lớn qua WebSocket. Thay vào đó, việc upload file được xử lý qua HTTP Multipart/form-data, sử dụng middleware (Multer) để stream trực tiếp lên Cloud CDN (Cloudinary). Backend trả về Asset URI, sau đó chỉ gửi URL này qua socket trong payload message, giúp tối ưu băng thông và hiệu năng hệ thống.

3.4 Luồng xử lý Message

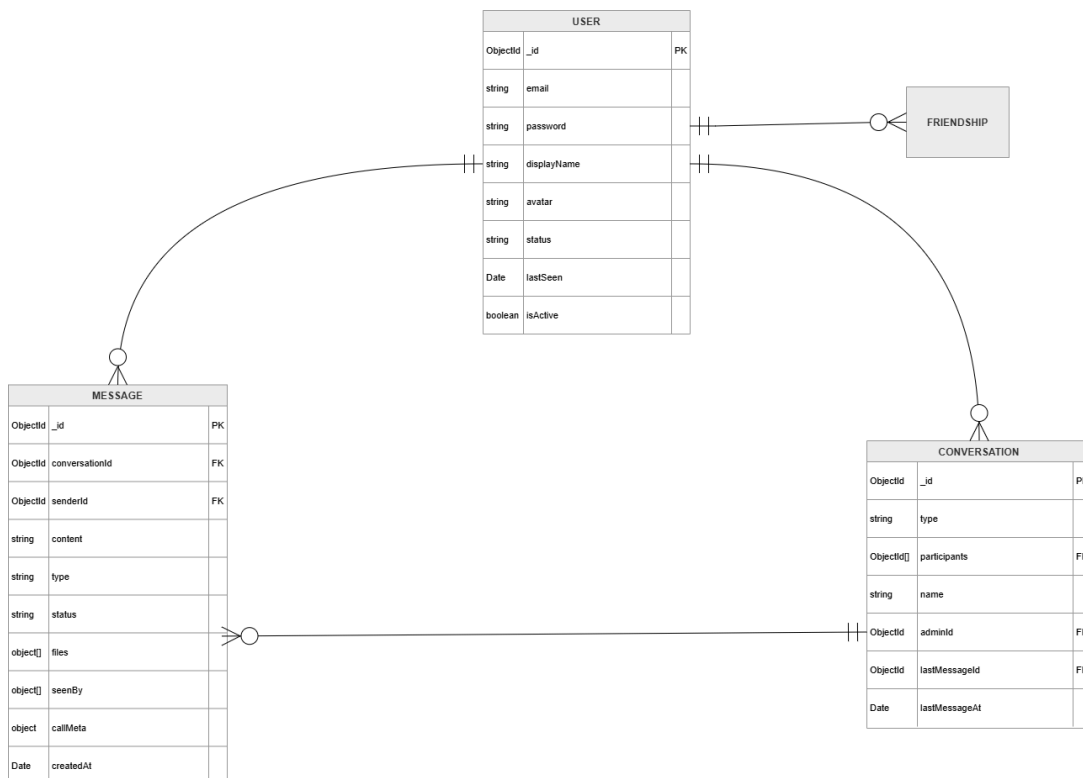


3.5 Luồng xử lý WebRTC & Signaling



CHƯƠNG 4. THIẾT KẾ CƠ SỞ DỮ LIỆU

4.1 Sơ đồ ERD



4.2 Nguyên tắc thiết kế và tối ưu dữ liệu

Thiết kế cơ sở dữ liệu được xây dựng nhằm đảm bảo hiệu năng và khả năng mở rộng trong môi trường phân tán:

- Tách Message thành collection độc lập: giúp tránh phình to document và hỗ trợ mở rộng tốt khi số lượng tin nhắn tăng, đồng thời tối ưu truy vấn phân trang.
- Sử dụng lastMessageId trong Conversation: cho phép truy xuất nhanh danh sách hội thoại mà không cần scan toàn bộ Message, giảm chi phí truy vấn.
- Sắp xếp theo timestamp (createdAt): đảm bảo thứ tự hiển thị ổn định trong môi trường bất đồng bộ, thay vì phụ thuộc vào sequence.
- Phân trang dữ liệu (Pagination): giới hạn số lượng bản ghi mỗi lần tải (~50) nhằm giảm tải hệ thống và cải thiện thời gian phản hồi.

4.3 Cơ chế caching và tối ưu truy vấn

Hệ thống sử dụng Redis như một lớp in-memory caching nhằm giảm tải cho cơ sở dữ liệu và tăng tốc độ truy xuất dữ liệu thời gian thực:

- Trạng thái hoạt động của người dùng cùng ánh xạ userId ↔ socketId được lưu trữ trong Redis dưới dạng Hash Map (hset, hget). Phục vụ hiệu quả cho định tuyến message và cập nhật trạng thái online/offline.
- Các dữ liệu mang tính tạm thời được lưu trực tiếp trên Redis, giúp hạn chế đáng kể số lượng truy vấn đọc/ghi lặp lại lên hệ thống lưu trữ chính.
- Lớp trung gian (Pub/Sub) giúp đồng bộ trạng thái và sự kiện giữa các backend instance, đảm bảo tính nhất quán khi hệ thống scale nhiều node.

CHƯƠNG 5. HIỆU NĂNG VÀ KHẢ NĂNG MỞ RỘNG

5.1 Phân phối kết nối

Hệ thống sử dụng Nginx làm Reverse Proxy và Load Balancer nhằm điều phối lưu lượng truy cập và tối ưu hiệu năng cho cả HTTP và WebSocket:

```
upstream backend_pool {
    least_conn;
    server backend:5051;
    keepalive 64;
}
```

- **least_conn**: phân phối request đến backend có số kết nối thấp nhất, giúp cân bằng tải hiệu quả trong môi trường nhiều kết nối đồng thời (đặc biệt với WebSocket).
- **keepalive 64**: duy trì kết nối TCP persistent với backend, giảm chi phí thiết lập kết nối mới và cải thiện độ trễ.

5.1.1 Xử lý HTTP API

```
location /api/ {
    proxy_pass http://backend_pool;

    proxy_http_version 1.1;
    proxy_set_header Host                $host;
    proxy_set_header X-Real-IP           $remote_addr;
    proxy_set_header X-Forwarded-For     $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto  $scheme;

    proxy_read_timeout    60s;
    proxy_connect_timeout 5s;
}
```

- Định tuyến toàn bộ REST API về backend cluster
- Forward các header cần thiết để backend xác định client
- Thiết lập timeout phù hợp cho request thông thường

5.1.2 Xử lý WebSocket

```
location /socket.io/ {
    proxy_pass http://backend_pool;

    proxy_http_version 1.1;

    proxy_set_header Upgrade    $http_upgrade;
    proxy_set_header Connection "upgrade";

    proxy_set_header Host        $host;
    proxy_set_header X-Real-IP    $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

    proxy_buffering    off;
    proxy_cache        off;

    proxy_read_timeout    3600s;
    proxy_send_timeout    3600s;
    proxy_connect_timeout 10s;
}
```

- Upgrade HTTP thành WebSocket thông qua header Upgrade và Connection
- proxy_buffering off: loại bỏ cơ chế buffer của Nginx, đảm bảo dữ liệu realtime không bị trì hoãn
- Timeout dài để duy trì kết nối WebSocket lâu dài (persistent connection)

5.1.3 Xử lý Front-end

```
location / {
    proxy_pass http://frontend;

    proxy_http_version 1.1;
    proxy_set_header Upgrade    $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host        $host;

    proxy_read_timeout 3600s;
}
```

- Định tuyến request frontend sang service riêng
- Tách biệt rõ frontend và backend giúp dễ scale độc lập

5.2 Đồng bộ realtime với Redis Pub/Sub

Hệ thống sử dụng `@socket.io/redis-adapter` để đồng bộ sự kiện giữa các backend instance:

```
export const pubClient = new Redis(REDIS_URL, {
  lazyConnect: true,
  maxRetriesPerRequest: 3,
  enableReadyCheck: false,
});

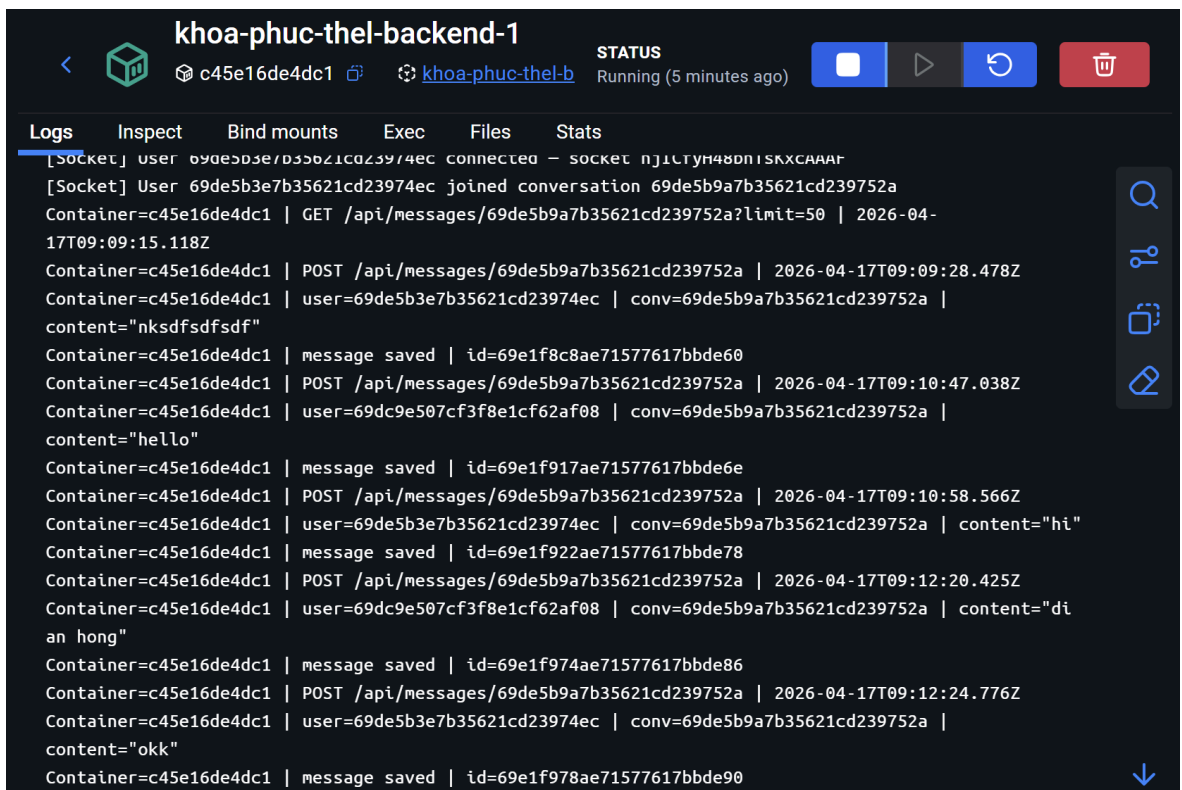
export const subClient = pubClient.duplicate();
```

- Backend instance A nhận sự kiện `message:send` từ Client A
- Event được publish vào Redis Pub/Sub
- Backend instance B (đang giữ kết nối với Client B) nhận event
- Server B gửi message đến đúng client đích

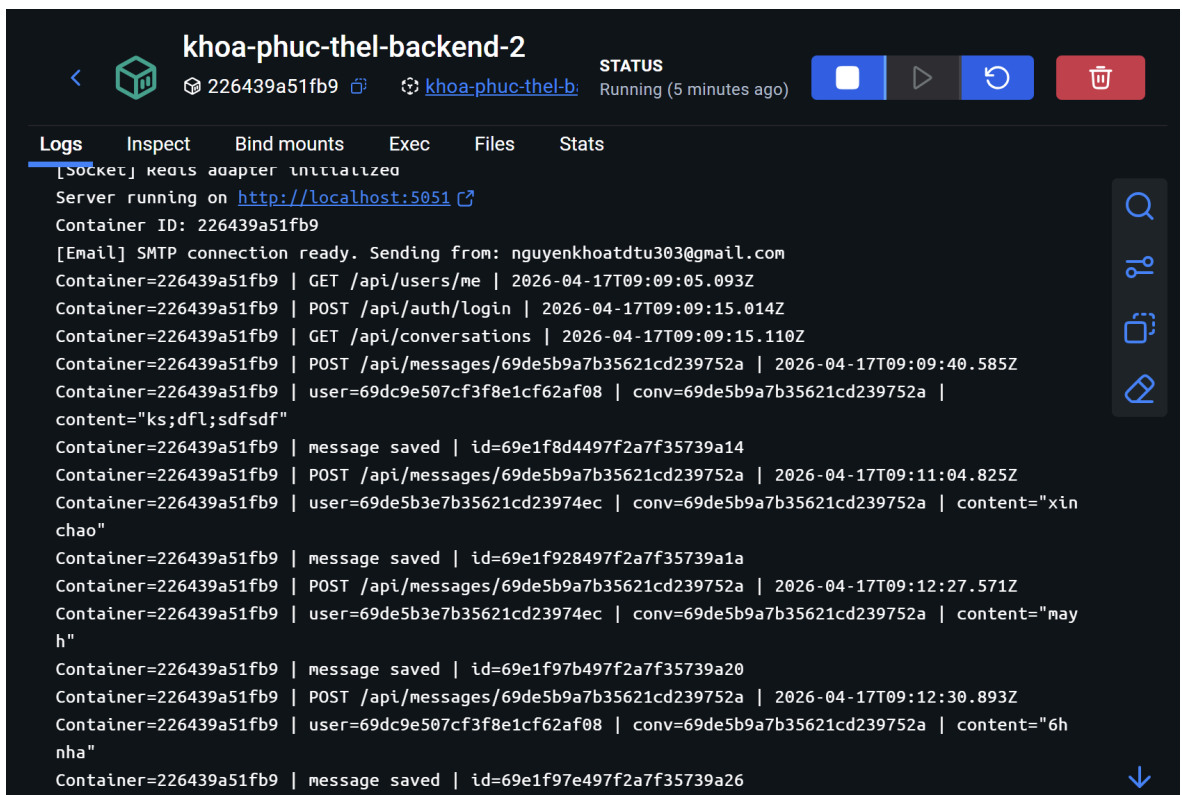
5.3 Minh chứng hệ thống scale

☐	▼	●	khoa-phuc-thel	-	-	-	☐	:	
☐		●	chat_redis	4fb1f110fff0	redis:7-alpi	6379:6379	☐	:	
☐		●	chat_mongo	6e112bae8d26	mongo:7		☐	:	
☐		●	chat_frontenc	2705f674fbcd	khoa-phuc-		☐	:	
☐		●	chat_nginx	116374a76d12	nginx:alpin	80:80	☐	:	
☐		●	backend-1	c45e16de4dc1	khoa-phuc-		☐	:	
☐		●	backend-2	226439a51fb9	khoa-phuc-		☐	:	

Hình 5.3.1 Hệ thống scale 2 backend



Hình 5.3.2 Hình ảnh gửi tin nhắn từ server 1



Hình 5.3.3 Hình ảnh gửi tin nhắn từ server 2

CHƯƠNG 6. BẢO MẬT

6.1 Xác thực

Hệ thống triển khai cơ chế xác thực dựa trên JSON Web Token (JWT), áp dụng thống nhất cho cả REST API và WebSocket:

- Các request HTTP được bảo vệ thông qua middleware xác thực JWT (Bearer Token) trong header, đảm bảo chỉ người dùng hợp lệ mới có thể truy cập tài nguyên hệ thống.
- Token được kiểm tra ngay tại bước connection handshake (`io.connect({ auth: { token } })`). Định danh người dùng được gán vào `socket.data.userId`, giúp ràng buộc phiên kết nối với đúng chủ thể và ngăn chặn hành vi giả mạo.

6.2 Bảo vệ hệ thống

Hệ thống sử dụng `express-rate-limit` kết hợp `rate-limit-redis` để kiểm soát số lượng request theo thời gian, nhằm hạn chế các cuộc tấn công spam và brute-force ở tầng ứng dụng. Thông tin xác thực được mã hóa bằng thuật toán băm Bcrypt, đảm bảo mật khẩu không bị lộ dưới dạng plaintext ngay cả khi xảy ra rò rỉ dữ liệu.

6.3 Bảo mật WebRTC

Hệ thống áp dụng cơ chế bảo mật native của WebRTC cho các kết nối đa phương tiện. Toàn bộ luồng Audio/Video được mã hóa đầu cuối (end-to-end) nhằm chống lại các tấn công nghe lén. Backend chỉ đóng vai trò Signaling Server (trao đổi SDP/ICE), không tham gia vào luồng truyền media. Dữ liệu đa phương tiện được truyền trực tiếp theo mô hình P2P, đảm bảo tính riêng tư và giảm thiểu rủi ro truy cập trái phép từ phía server.

CHƯƠNG 7. KẾT LUẬN

7.1 Kết luận

Hệ thống đã xây dựng thành công nền tảng giao tiếp thời gian thực với kiến trúc phân tán, đáp ứng tốt các yêu cầu về độ trễ thấp, khả năng mở rộng và tính sẵn sàng.

Việc kết hợp WebSocket, Redis Pub/Sub và WebRTC giúp tối ưu hiệu năng cho cả dữ liệu văn bản và đa phương tiện, đồng thời giảm tải cho server thông qua mô hình P2P. Kiến trúc Stateless cùng Docker và Nginx cho phép hệ thống dễ dàng mở rộng theo chiều ngang và vận hành linh hoạt.

7.2 Hướng phát triển

- Triển khai Kubernetes để tự động hóa việc triển khai, scaling và quản lý container ở quy mô lớn.
- Tích hợp Kafka / RabbitMQ cho xử lý bất đồng bộ cho các tác vụ lớn.
- Bổ sung Push Notification, tích hợp thông báo thời gian thực.

TÀI LIỆU THAM KHẢO

Tiếng Việt

...

Tiếng Anh

OpenJS Foundation. (2024). *Express - Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com>

Mongoose. (2024). *Mongoose - Elegant MongoDB object modeling for Node.js*. <https://mongoosejs.com>

Socket.IO. (2024). *Socket.IO - Bidirectional and low-latency communication for every platform*. <https://socket.io>

Meta Platforms, Inc.. (2024). *React - The library for web and native user interfaces*. <https://react.dev>

Poimandres. (2024). *Zustand - Bear necessities for state management in React*. <https://zustand-demo.pmnd.rs/>

Remix Software. (2024). *React Router - Declarative routing for React applications*. <https://reactrouter.com>

Redis. (2024). *Redis - The open source, in-memory data store used by millions of developers as a database, cache, streaming engine, and message broker*. <https://redis.io>

Docker Inc.. (2024). *Docker - Accelerated container application development*. <https://www.docker.com>

Nginx, Inc.. (2024). *NGINX - High performance load balancer, web server, & reverse proxy*. <https://nginx.org>